



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: Pablo Sebastian Noyola Torres

N° de Cuenta: 320023213

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 07

SEMESTRE 2027 - 1

FECHA DE ENTREGA LÍMITE: 09 de septiembre de 2026

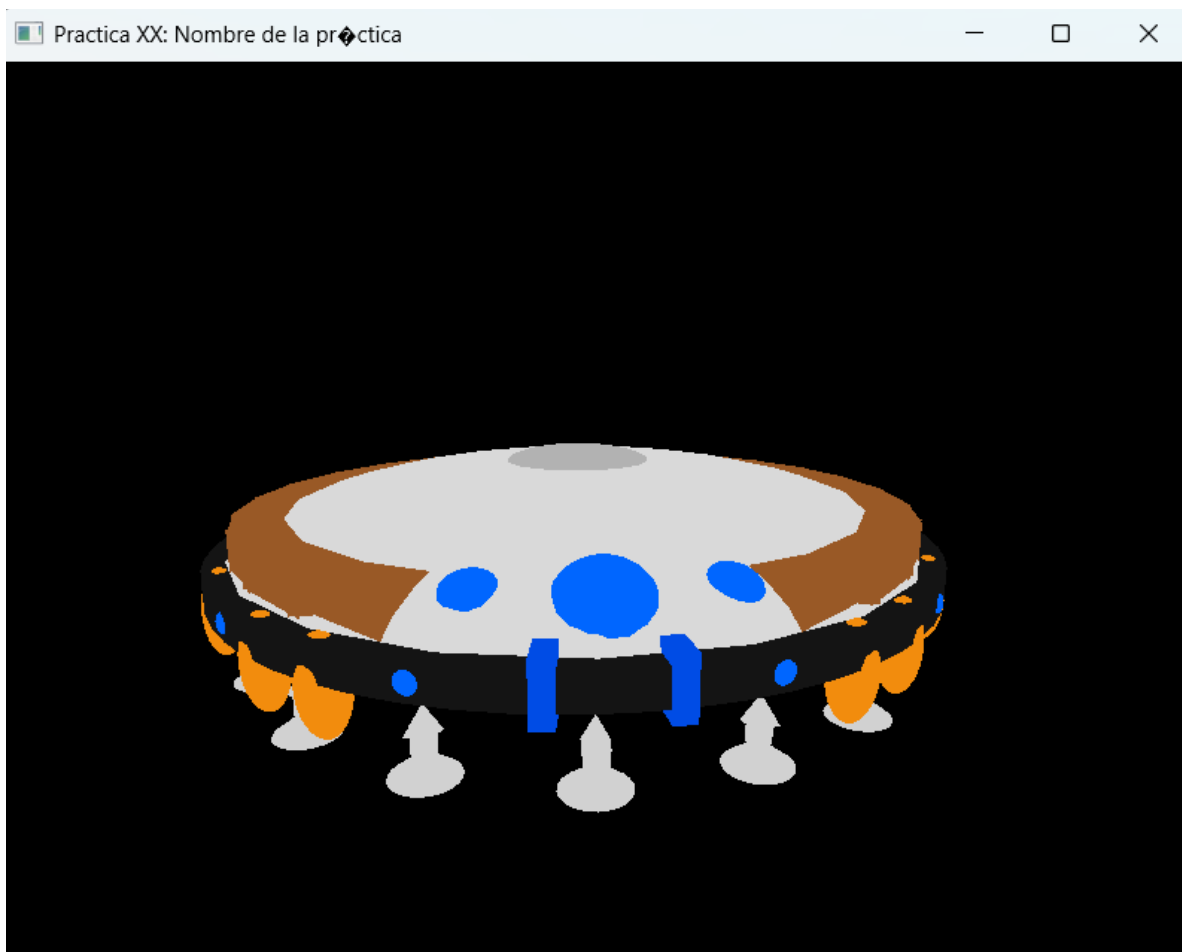
CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.- Crear un cohete espacial con instancias de: pirámides, cilindros, esferas, conos, y cubos.

Ejecución:



Enlace de video de ejecución: <https://www.youtube.com/watch?v=9d1hHF0cr3E>

Código de ejercicio:

```

383 // -----Cohete Espacial -----
384 // Cuerpo principal de la nave
385 model = glm::mat4(1.0f);
386 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f)); // Posición central en la escena
387 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
388 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
389 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
390 glm::mat4 modelCuerpo = model;
391 model = glm::scale(model, glm::vec3(3.0f, 0.8f, 3.0f)); // Aumentamos X y Z de 2.5f a 3.0f
392 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
393 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
394 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
395 color = glm::vec3(0.85f, 0.85f, 0.85f);
396 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
397 sp.render();
398
399 // Franja de cubos negros que rodea el cuerpo principal de la nave
400 int numCubosFranja = 60;
401 float radioFranja = 2.85f;
402 for (int i = 0; i < numCubosFranja; i++) {
403     float angulo = i * (2.0f * 3.1416f / numCubosFranja);
404     float x = cos(angulo) * radioFranja;
405     float z = sin(angulo) * radioFranja;
406     model = modelCuerpo;
407     model = glm::translate(model, glm::vec3(x, 0.0f, z));
408     model = glm::rotate(model, -angulo, glm::vec3(0.0f, 1.0f, 0.0f));
409     model = glm::scale(model, glm::vec3(0.55f, 0.25f, 0.35f));
410     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
411     color = glm::vec3(0.08f, 0.08f, 0.08f);
412     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
413     meshList[0]-->RenderMesh();
414 }
415
416 // Accesorios naranjas de la nave
417 float radioAnillo = 3.00f;
418 float radioEsferas = radioAnillo + 0.05f;
419 int numNaranjas = 16;
420 for (int i = 0; i < numNaranjas; i++) {

```

```

421         float angulo = i * (2.0f * 3.1416f / numNaranjas);
422
423         if (angulo > 5.60f || angulo < 0.68f) continue;
424         float x = cos(angulo) * radioAnillo;
425         float z = sin(angulo) * radioAnillo;
426         model = modelCuerpo;
427         model = glm::translate(model, glm::vec3(x, -0.20f, z));
428         model = glm::rotate(model, -angulo, glm::vec3(0.0f, 1.0f, 0.0f));
429         model = glm::scale(model, glm::vec3(0.15f, 0.35f, 0.21f));
430         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
431         color = glm::vec3(0.95f, 0.55f, 0.05f);
432         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
433         sp.render();
434     }
435
436     // Ventanas de cubos azules de la nave
437     float angulosVentanas[2] = { -0.15f, 0.10f };
438     for (int i = 0; i < 2; i++) {
439         float angulo = angulosVentanas[i];
440         float x = cos(angulo) * (radioAnillo + 0.02f);
441         float z = sin(angulo) * (radioAnillo + 0.02f);
442
443         model = modelCuerpo;
444         model = glm::translate(model, glm::vec3(x, 0.0f, z));
445         model = glm::rotate(model, -angulo, glm::vec3(0.0f, 1.0f, 0.0f));
446         model = glm::scale(model, glm::vec3(0.28f, 0.42f, 0.15f));
447
448         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
449         color = glm::vec3(0.0f, 0.3f, 0.9f);
450         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
451         meshList[0]->RenderMesh();
452     }
453
454     // Accesorios de la nave de esferas azules
455     float angulosEsferas[] = {
456         -0.35f,
457         0.35f,
458         0.90f, 1.30f, 1.70f, 2.10f, 2.50f, 2.90f, 3.30f, 3.70f, 4.10f, 4.50f, 4.90f, 5.35f

```

```

458         0.90f, 1.30f, 1.70f, 2.10f, 2.50f, 2.90f, 3.30f, 3.70f, 4.10f, 4.50f, 4.90f, 5.35f
459     };
460
461     for (float angulo : angulosEsferas) {
462         float x = cos(angulo) * radioEsferas;
463         float z = sin(angulo) * radioEsferas;
464         model = modelCuerpo;
465         model = glm::translate(model, glm::vec3(x, 0.0f, z));
466         model = glm::rotate(model, -angulo, glm::vec3(0.0f, 1.0f, 0.0f));
467         model = glm::scale(model, glm::vec3(0.11f, 0.11f, 0.11f));
468         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
469         color = glm::vec3(0.0f, 0.4f, 1.0f);
470         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
471         sp.render();
472     }
473
474     // Para accesorios naranjas de la nave cercanos a esferas de lado de ventanas de cubos
475
476     float angulosNaranjasFrontales[2] = { -0.58f, 0.58f };
477     for (float angulo : angulosNaranjasFrontales) {
478         float x = cos(angulo) * radioAnillo;
479         float z = sin(angulo) * radioAnillo;
480
481         model = modelCuerpo;
482         model = glm::translate(model, glm::vec3(x, -0.20f, z));
483         model = glm::rotate(model, -angulo, glm::vec3(0.0f, 1.0f, 0.0f));
484         model = glm::scale(model, glm::vec3(0.15f, 0.35f, 0.21f));
485
486         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
487         color = glm::vec3(0.95f, 0.55f, 0.05f);
488         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
489         sp.render();
490     }
491
492     // Para patas de la nave
493     int numPatas = 16;
494     float radioBasePatas = 2.75f;
495

```

```

496     for (int i = 0; i < numPatas; i++) {
497         float angulo = i * (2.0f * 3.1416f / numPatas);
498         float x = cos(angulo) * radioBasePatas;
499         float z = sin(angulo) * radioBasePatas;
500         color = glm::vec3(0.82f, 0.82f, 0.82f);
501         model = modelCuerpo;
502         model = glm::translate(model, glm::vec3(x, -0.42f, z));
503         model = glm::rotate(model, -angulo, glm::vec3(0.0f, 1.0f, 0.0f));
504
505         glm::mat4 modelArticulacion = model;
506         glm::mat4 modelAnclaje = glm::scale(model, glm::vec3(0.18f, 0.2f, 0.18f));
507         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelAnclaje));
508         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
509         meshList[4]->RenderMesh();
510
511         model = modelArticulacion;
512         model = glm::translate(model, glm::vec3(0.0f, -0.15f, 0.0f));
513         glm::mat4 modelMuslo = model;
514
515         glm::mat4 modelCilindroMuslo = glm::scale(model, glm::vec3(0.09f, 0.2f, 0.09f));
516         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelCilindroMuslo));
517         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
518         meshList[2]->RenderMeshGeometry();
519
520         model = modelMuslo;
521         model = glm::translate(model, glm::vec3(0.0f, -0.15f, 0.0f));
522         glm::mat4 modelPuntaConica = glm::scale(model, glm::vec3(0.12f, 0.18f, 0.12f));
523         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPuntaConica));
524         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
525         meshList[3]->RenderMeshGeometry();
526     }
527
528     // Para 3 esferas de decoracion arriba de la nave
529     float radioCúpulaTop = 2.5f;
530
531     float angulosEsferasSup[3] = { -0.025f, -0.35f, 0.30f };
532     float escalasEsferasSup[3] = { 0.38f, 0.28f, 0.28f };
533

```

```

534     for (int i = 0; i < 3; i++) {
535         float angulo = angulosEsferasSup[i];
536         float x = cos(angulo) * radioCúpulaTop;
537         float z = sin(angulo) * radioCúpulaTop;
538
539         model = modelCuerpo;
540         model = glm::translate(model, glm::vec3(x, 0.20f, z));
541         model = glm::rotate(model, -angulo, glm::vec3(0.0f, 1.0f, 0.0f));
542         model = glm::scale(model, glm::vec3(escalasEsferasSup[i]));
543
544         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
545
546         glm::vec3 colorAzul = glm::vec3(0.0f, 0.4f, 1.0f);
547         glUniform3fv(uniformColor, 1, glm::value_ptr(colorAzul));
548
549         sp.render();
550     }
551
552     // Para paneles cafe de arriba de la nave
553     int totalPaneles = 26;
554     float radioCafes = 2.5f;
555     float anguloMin = -2.9f;
556     float anguloMax = 2.9f;
557
558     for (int i = 0; i < totalPaneles; i++) {
559         float t = (float)i / (totalPaneles - 1);
560         float angulo = anguloMin + t * (anguloMax - anguloMin);
561
562         if (angulo > -0.55f && angulo < 0.55f) continue;
563
564         float x = cos(angulo) * radioCafes;
565         float z = sin(angulo) * radioCafes;
566
567         model = modelCuerpo;
568         model = glm::translate(model, glm::vec3(x, 0.25f, z));
569         model = glm::rotate(model, -angulo, glm::vec3(0.0f, 1.0f, 0.0f));
570
571         model = glm::scale(model, glm::vec3(0.8f, 0.37f, 0.8f));

```

```

570
571         model = glm::scale(model, glm::vec3(0.8f, 0.37f, 0.8f));
572
573         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
574
575         glm::vec3 colorCafé = glm::vec3(0.6f, 0.35f, 0.15f);
576         glUniform3fv(uniformColor, 1, glm::value_ptr(colorCafé));
577         meshList[0]->RenderMesh();
578     }
579
580     // Para escotilla de arriba de la nave
581     model = modelCuerpo;
582     model = glm::translate(model, glm::vec3(0.0f, 0.75f, 0.0f));
583     model = glm::scale(model, glm::vec3(0.65f, 0.15f, 0.65f));
584     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
585     glm::vec3 colorEscotilla = glm::vec3(0.70f, 0.70f, 0.70f);
586     glUniform3fv(uniformColor, 1, glm::value_ptr(colorEscotilla));
587     sp.render();
588

```

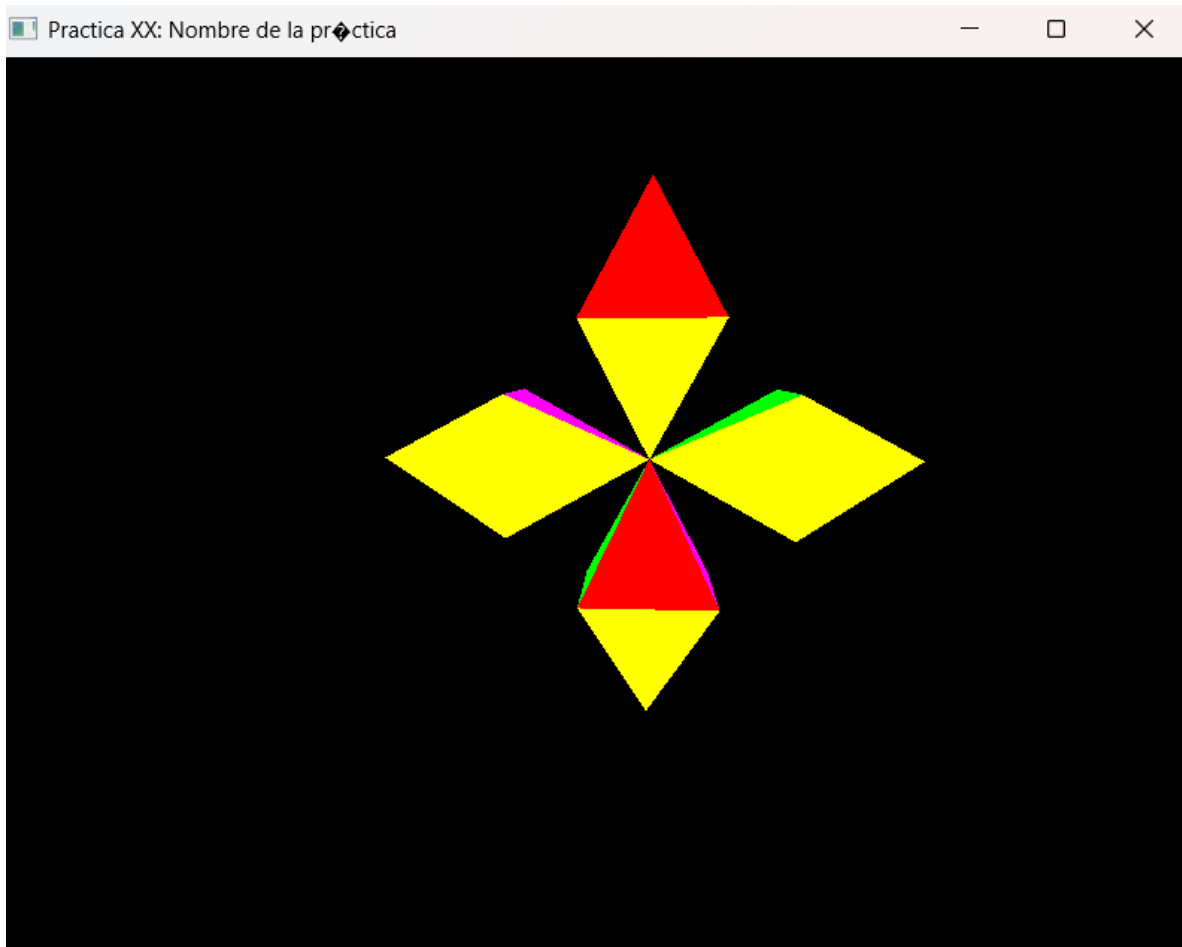
Para este primer ejercicio de la realización de un cohete espacial con instancias de: pirámides, cilindros, esferas, conos, y cubos, me inspire en la nave de Freezer de la serie anime dragon ball z, estructurando toda la geometría mediante transformaciones matriciales en OpenGL con GLM (translate, rotate, scale) para posicionar cada componente de manera simétrica, comenzando por definir un cuerpo principal gris claro, una franja perimetral de cubos negros, accesorios naranjas y ventanas de cubos azules distribuidos radialmente con funciones trigonométricas, continuando con las patas de soporte formadas por pirámides, cilindros y conos, y finalizando en la parte superior decorada con esferas azules, paneles cafés y una escotilla central circular hecha con un cilindro.

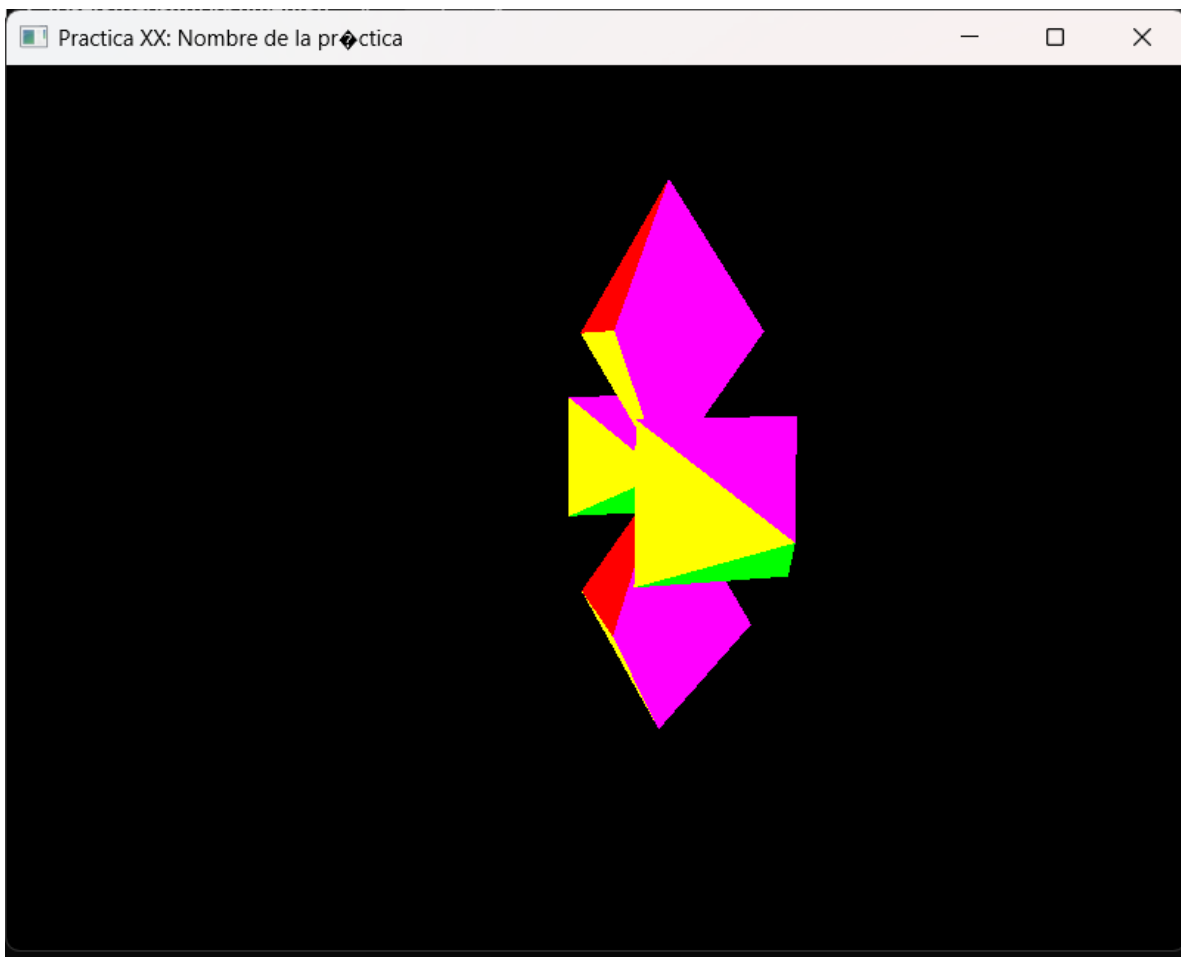
Imagen de modelo basado para la realización de la nave:

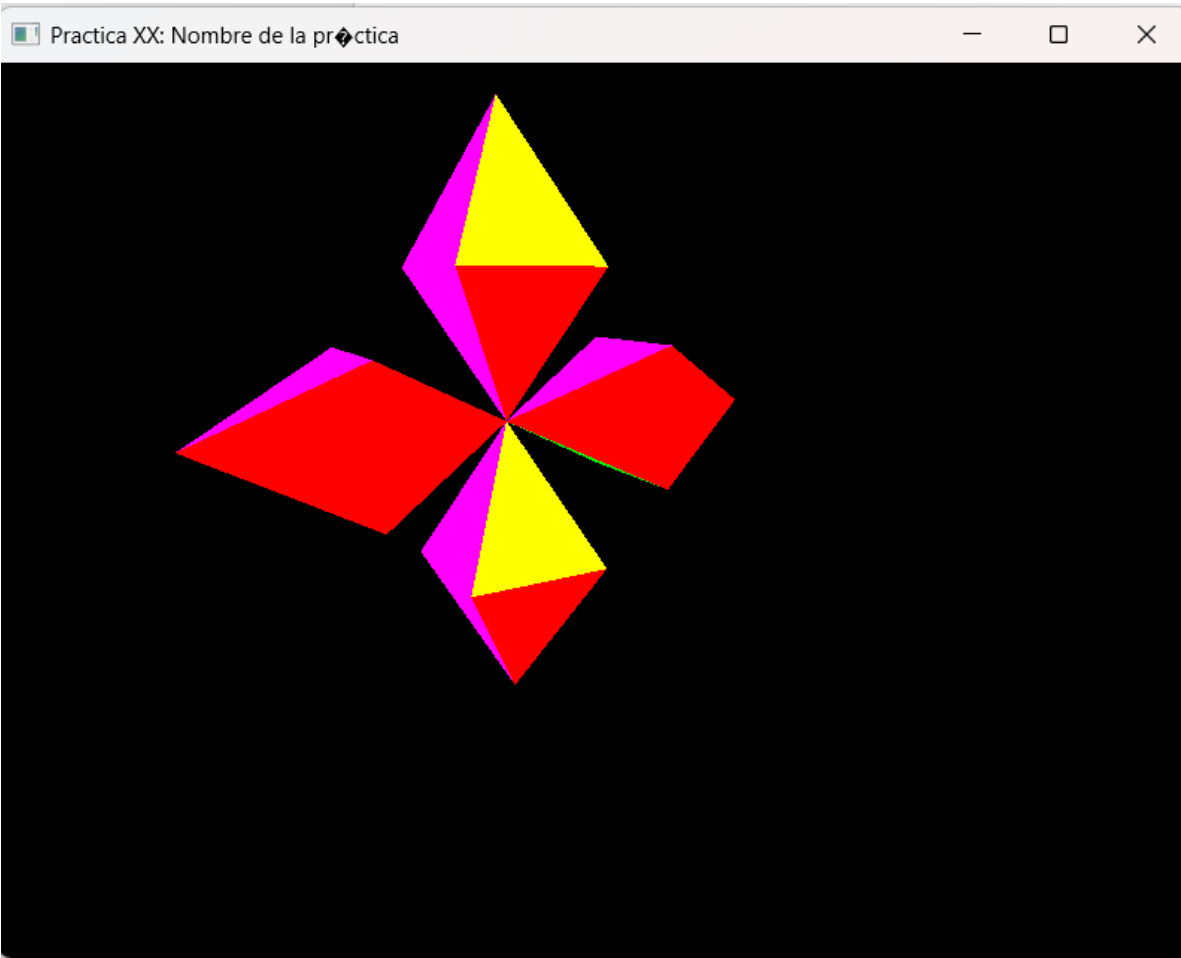


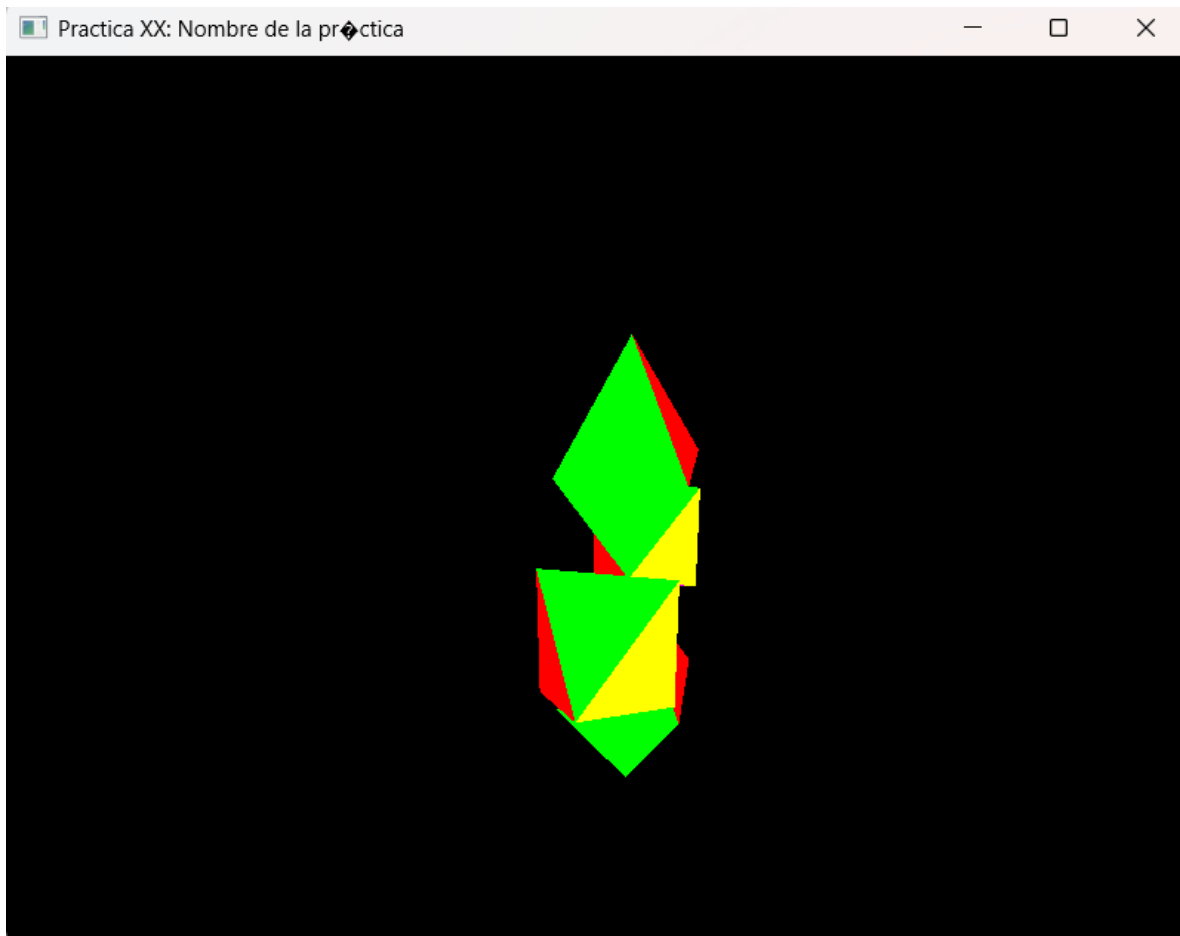
2.- Generar la figura de unión de 8 pirámides como la que se adjunta. Cada cara de cada pirámide es de un color diferente considerando que al ver una sola pirámide desde todos los ángulos se verían las caras triangulares de color: roja, verde, amarilla, magenta. Y la cara cuadrada se ve de color azul.

Ejecución:









Enlace de video de ejecución: <https://youtu.be/TOoYNON4QnU>

Código de ejercicio:

```

593 // -----Figura de unión de 8 piramides -----
594 // Para dibujar cada cara de color diferente de la piramide
595 auto renderPiramideMulticolor = [&](glm::mat4& baseModel) {
596     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
597     glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
598
599     // Cara 1 - Roja
600     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(baseModel));
601     glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(1.0f, 0.0f, 0.0f)));
602     meshList[5]-->RenderMesh();
603
604     // Cara 2 - Verde
605     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(baseModel));
606     glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 1.0f, 0.0f)));
607     meshList[6]-->RenderMesh();
608
609     // Cara 3 - Amarilla
610     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(baseModel));
611     glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(1.0f, 1.0f, 0.0f)));
612     meshList[7]-->RenderMesh();
613
614     // Cara 4 - Magenta
615     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(baseModel));
616     glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(1.0f, 0.0f, 1.0f)));
617     meshList[8]-->RenderMesh();
618
619     // Cara cuadrada (Base) - Azul
620     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(baseModel));
621     glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 1.0f)));
622     meshList[9]-->RenderMesh();
623     };
624
625 // Piramides de arriba-----
626 // Piramide inferior

```

```

624
625 // Piramides de arriba-----
626 // Piramide inferior
627
628 model = glm::mat4(1.0f);
629 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
630 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
631 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
632 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
633 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
634 renderPiramideMulticolor(model);
635
636 // Piramide superior
637 model = glm::mat4(1.0f);
638 model = glm::translate(model, glm::vec3(0.0f, 1.0f, -4.0f));
639 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
640 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
641 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
642 renderPiramideMulticolor(model);
643
644 // Piramides de abajo-----
645 // Piramide superior
646 model = glm::mat4(1.0f);
647 model = glm::translate(model, glm::vec3(0.0f, -1.0f, -4.0f));
648 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
649 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
650 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
651 renderPiramideMulticolor(model);
652
653 // Piramide inferior
654 model = glm::mat4(1.0f);
655 model = glm::translate(model, glm::vec3(0.0f, -2.0f, -4.0f));
656 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
657 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
658 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
659 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
660 renderPiramideMulticolor(model);
661

```

```

659     model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
660     renderPiramideMulticolor(model);
661
662     // Piramides de izquierda-----
663     // Piramide inferior
664     model = glm::mat4(1.0f);
665     model = glm::translate(model, glm::vec3(-0.5f, -0.5f, -4.0f));
666     model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
667     model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
668     model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
669     model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
670     model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
671     renderPiramideMulticolor(model);
672
673     // Piramide superior
674     model = glm::mat4(1.0f);
675     model = glm::translate(model, glm::vec3(-1.5f, -0.5f, -4.0f));
676     model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
677     model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
678     model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
679     model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
680     model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
681     renderPiramideMulticolor(model);
682
683     // Piramides de derecha-----
684     // Piramide inferior
685     model = glm::mat4(1.0f);
686     model = glm::translate(model, glm::vec3(0.5f, -0.5f, -4.0f));
687     model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
688     model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
689     model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
690     model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
691     model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
692     renderPiramideMulticolor(model);
693
694     // Piramide superior
695     model = glm::mat4(1.0f);
696     model = glm::translate(model, glm::vec3(1.5f, -0.5f, -4.0f));

```

```

682
683     // Piramides de derecha-----
684     // Piramide inferior
685     model = glm::mat4(1.0f);
686     model = glm::translate(model, glm::vec3(0.5f, -0.5f, -4.0f));
687     model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
688     model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
689     model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
690     model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
691     model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
692     renderPiramideMulticolor(model);
693
694     // Piramide superior
695     model = glm::mat4(1.0f);
696     model = glm::translate(model, glm::vec3(1.5f, -0.5f, -4.0f));
697     model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
698     model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
699     model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
700     model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
701     model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
702     renderPiramideMulticolor(model);
703
704
705
706     glUseProgram(0);
707     mainWindow.swapBuffers();
708 }
709 return 0;
710 }
711

```

```

316 int main()
317 {
318     mainWindow = Window(800, 600);
319     mainWindow.Initialise();
320     //Cilindro y cono reciben resolución (slices, rebanadas) y Radio de circunferencia de la base y tapa
321
322     CrearCubo();//índice 0 en MeshList
323     CrearPiramideTriangular();//índice 1 en MeshList
324     CrearCilindro(5, 1.0f);//índice 2 en MeshList
325     CrearCono(25, 2.0f);//índice 3 en MeshList
326     CrearPiramideCuadrangular();//índice 4 en MeshList
327     CrearCarasPiramideCuadrangular(); // Para caras separadas
328     CreateShaders();
329

```

```

132
133 // Función agregada para separar cada cara de una piramide cuadrangular
134 void CrearCarasPiramideCuadrangular()
135 {
136     GLfloat v[] = {
137         0.5f, -0.5f, 0.5f,
138         0.5f, -0.5f, -0.5f,
139         -0.5f, -0.5f, -0.5f,
140         -0.5f, -0.5f, 0.5f,
141         0.0f, 0.5f, 0.0f
142     };
143
144     // Caras
145     unsigned int ind_t1[] = { 0, 3, 4 }; // Cara 1
146     unsigned int ind_t2[] = { 3, 2, 4 }; // Cara 2
147     unsigned int ind_t3[] = { 2, 1, 4 }; // Cara 3
148     unsigned int ind_t4[] = { 1, 0, 4 }; // Cara 4
149     unsigned int ind_base[] = { 0, 1, 2, 0, 2, 3 }; // Cara cuadrada (Base)
150
151     Mesh* cara1 = new Mesh(); cara1->CreateMesh(v, ind_t1, 15, 3); meshList.push_back(cara1);
152     Mesh* cara2 = new Mesh(); cara2->CreateMesh(v, ind_t2, 15, 3); meshList.push_back(cara2);
153     Mesh* cara3 = new Mesh(); cara3->CreateMesh(v, ind_t3, 15, 3); meshList.push_back(cara3);
154     Mesh* cara4 = new Mesh(); cara4->CreateMesh(v, ind_t4, 15, 3); meshList.push_back(cara4);
155     Mesh* base = new Mesh(); base->CreateMesh(v, ind_base, 15, 6); meshList.push_back(base);
156 }
157

```

Para este segundo ejercicio enfocado en la unión de 8 pirámides con caras de distintos colores, implementé una función llamada `CrearCarasPiramideCuadrangular()` para definir los vértices base y separar los índices de cada componente geométrica, almacenando individualmente las cuatro caras triangulares y la base cuadrada en la lista de mallas, y adicionalmente utilicé una expresión lambda llamada `renderPiramideMulticolor` que captura el contexto y recibe una matriz modelo base para aplicar las transformaciones de proyección y vista, permitiéndome renderizar cada cara con un color distinto rojo, verde, amarillo y magenta para los triángulos, y azul para la base mediante llamadas reutilizables con las rotaciones y traslaciones necesarias para ensamblar toda la figura geométrica.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

El principal problema fue el tiempo y para el segundo ejercicio considero que no comprendí del todo la forma de la figura a realizar en la imagen, sin embargo fuera de eso, no se presentó ningún problema mayor.

3.- Conclusión:

- a. Los ejercicios del reporte: Complejidad, Explicación.
- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica
- c. Conclusión

Tras la realización de esta practica considero que los ejercicios solicitados fueron de gran ayuda para comprender de mejor manera los conceptos vistos en clase y tuvo una alta complejidad pues en lo personal me costo mucho tiempo realizarla. Por otra parte, considero que no se si entendí correctamente la forma de la figura solicitada en el ejercicio 2. Finalmente, me parece que fue una muy buena práctica.

1. Bibliografía en formato APA

Cppreference. (s.f.). Lambda expressions (desde C++11). Recuperado de <https://en.cppreference.com/w/cpp/language/lambda>

The Khronos Group. (2017). OpenGL® 4.6 API Reference Pages (Documentación oficial de referencia de la API de OpenGL). <https://www.khronos.org/registry/OpenGL-Refpages/>

Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). OpenGL programming guide: The official guide to learning OpenGL, version 4.3 (8th ed.). Addison-Wesley.